

PITBULL: Temporal Safety Skill for Coding Agents

Pavel Marian¹, Stanislav Chumakov¹, Danil Ezhov¹, Alexander Boukhanovsky¹

¹AI Institute, ITMO University, Saint Petersburg, Russian Federation
svchumakov@itmo.ru, doezhov@itmo.ru, boukhanovsky@mail.ifmo.ru

Abstract

We present PITBULL, an executable temporal-safety skill for agent-generated feature programs. The skill detects point-in-time violations by comparing pipeline outputs on full and temporally truncated databases, adds canary perturbations for late-arriving fields, and validates every agent repair on held-out prediction times. Across three runs over 62 leaking programs, 97.8% of program–run pairs were repaired within five iterations. Packaged as a skill that an independent coding agent selects on its own, the same loop repairs 76 of 76 programs, matching direct invocation of the procedure.

Demo — <https://pavelfmarian.github.io/pitbull/demo/temporal-safety/>

Code — <https://github.com/pavelfmarian/pitbull>

Introduction and Related Work

Successful execution of a relational feature pipeline by a coding agent does not guarantee its temporal correctness. A pipeline that filters on a single reference timestamp can still read information that did not exist at the prediction time, because availability is spread across related tables and columns. Such violations yield plausible features and inflated offline results that do not reproduce at serving time. The property that every feature computed at time t depends only on facts available at time t is called point-in-time correctness.

Temporal leakage is a recurring source of irreproducible machine-learning results (Kaufman et al. 2012; Kapoor and Narayanan 2022), including in temporally split process-mining benchmarks (Weytjens and Weerdt 2021). The relational setting makes it harder: features are computed across several tables (Robinson et al. 2024), and the availability of a value may be governed by another row or by a column-specific timestamp. Static notebook analysis and pipeline instrumentation target preprocessing and train–test contamination (Yang et al. 2022; Grafberger et al. 2021). LLM-based code inspection is probabilistic (Alturayef and Hasine 2025) and insufficient as the only safeguard.

The PITFALL system introduced black-box differential execution for detecting and localising point-in-time violations (Chumakov 2025). PITBULL extends it from diagnosis

to a closed agentic loop of detection, repair, and independent revalidation. The verification is one-sided, in line with the view of look-ahead freedom as temporal non-interference: an observed divergence is a finite witness of a violation, and the absence of divergence at finitely many probes is no proof (Fonseca 2026).

System Overview

PITBULL is a skill that a coding agent invokes over a candidate feature program $\varphi(D, e, t)$: a database D , entities e , and a prediction time t . The agent writes and revises the program; the skill only executes it and returns evidence (Figure 1).

Differential witness. For each prediction time the skill runs the program twice, on the full database and on the database truncated at t :

$$y_{\text{full}} = \varphi(D, e, t), \quad y_{\text{past}} = \varphi(D|_t, e, t). \quad (1)$$

Any difference beyond a numerical tolerance is a LEAK witness, and the skill names the affected output columns. Equality only says that no violation was observed at the times tested.

Canary. Some channels are hidden by aggregation or exercised weakly by real future values. The skill therefore builds a counterfactual future: numbers are shifted, categories replaced by a reserved value, dates moved beyond the data horizon, including late-filled fields such as delivery dates. An output that reacts to this perturbation is reported as a canary hit. Canary is less specific than the witness (it fired on 4 of 30 clean control programs), so the report lists canary columns separately for a human to judge.

Repair loop and held-out gate. On a LEAK, the agent rewrites the program under a fixed natural-language instruction that names the class of error (features must use only information that exists at t ; reviews and delivery dates arrive after the purchase) and reruns the check. A candidate is CLEAN only if it executes and shows neither witness nor canary divergence on all development and held-out prediction times; missing code, runtime errors and timeouts count as failures. The loop stops at the first CLEAN or after five repairs. The full history of candidates and their verdicts is kept in a state file next to the code. A CLEAN verdict is scoped to the declared availability map and the tested times.

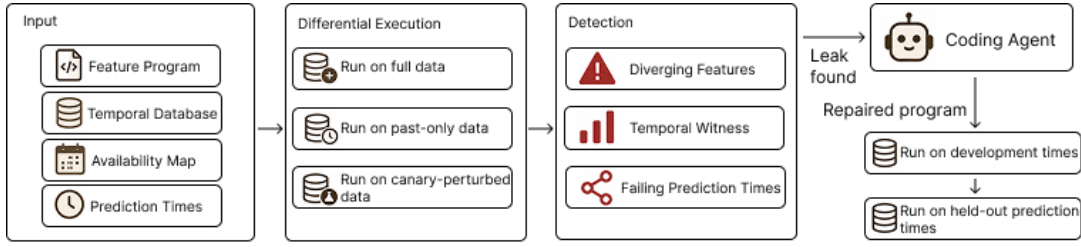


Figure 1: PITBULL architecture: the agent submits a feature program; the skill runs it on the full, the truncated, and the canary-perturbed database, reports diverging features, and gates each repair on development and held-out prediction times.

Invocation	n	CLEAN@1	CLEAN@5	Calls	Cost
Direct, 1 run	76	85.5%	98.7%	91	\$0.12
Direct, 3 runs	62×3	84.4%	97.8%	255	\$0.32
Agent-selected skill	76	98.7%	100%	578	\$0.48

Table 1: Repair outcomes with the same repair model. The skill row uses the same 76 programs as the first row; exact McNemar over paired per-program outcomes: 0 vs. 1 discordant, $p = 1.0$.

Demonstration

The demonstration runs in the browser at the linked page. Visitors pick an Olist profile and a candidate feature program (the bundled sample or their own file), set a development prediction time, and press *Run temporal check*. The page shows the three data views side by side, the diverging product features, the availability channel that caused the divergence (for the sample, reviews joined without a `review_creation_date` cut), and the acceptance gate on development and held-out times. A repair by the coding agent is then applied, re-checked, and offered for download as a patched source, a diff, and a JSON evidence report. Four study panels let visitors browse the evidence behind Table 1: CLEAN@ k across the corpus, a before/after view of one repair, the three repeated runs, and the number of prediction times needed to expose a leak. A live run takes under a minute per iteration. The same skill loads into a coding agent unchanged (Table 1, last row).

Experimental Snapshot

Table 1 summarises repair outcomes on leaking relational feature programs derived from Olist tasks, with one repair model. In three independent direct runs over 62 programs, 182 of 186 program–run pairs became clean within five iterations. A density experiment shows that two prediction times give 97.6% conservative witness detection; the witness had no false positives on 30 clean control programs, the canary fired on four.

We then packaged the loop as a reusable skill and ran it under an independent agentic harness that selects and invokes the skill on its own; the skill name does not appear in the task prompt. On the same 76 programs the skill matches direct invocation: CLEAN@5 reaches 100%, and the one discordant program is the single failure of direct invocation,

repaired at the first iteration under the skill. In skill mode the agent also sees the checker output with column names, which explains part of the gain at CLEAN@1. The price is packaging overhead: about $4 \times$ the inference cost and $6 \times$ the model calls per program, driven by the harness system prompt on every turn. The evidence is strongest for review-based leakage, which dominates the corpus.

Conclusion

PITBULL gives a coding agent an executable way to detect, localise, repair, and revalidate point-in-time violations in relational feature pipelines, with the verdict made by the checker and the repair by the agent. Future work: independently annotated per-column availability maps, a corpus beyond review-based leakage, and automatic inference of availability maps under the same evidentiary standard.

References

- Alturayef, N.; and Hassine, J. 2025. Data leakage detection in machine learning code: transfer learning, active learning, or low-shot prompting? *PeerJ Computer Science*, 11: e2730.
- Chumakov, S. 2025. pitfall. <https://github.com/stas1f1/pitfall>.
- Fonseca, X. 2026. Look-Ahead-Freedom as Temporal Non-Interference: A Verifiable Correctness Property for Backtesting and Agentic Trading Pipelines. arXiv:2607.04958.
- Grafberger, S.; Guha, S.; Stoyanovich, J.; and Schelter, S. 2021. MLINSPECT: A Data Distribution Debugger for Machine Learning Pipelines. In *Proceedings of the 2021 International Conference on Management of Data, SIGMOD ’21*, 2736–2739. New York, NY, USA: Association for Computing Machinery. ISBN 9781450383431.
- Kapoor, S.; and Narayanan, A. 2022. Leakage and the Reproducibility Crisis in ML-based Science. arXiv:2207.07048.
- Kaufman, S.; Rosset, S.; Perlich, C.; and Stitelman, O. 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data*, 6(4).
- Robinson, J.; Ranjan, R.; Hu, W.; Huang, K.; Han, J.; Dobles, A.; Fey, M.; Lenssen, J. E.; Yuan, Y.; Zhang, Z.; He, X.; and Leskovec, J. 2024. RelBench: A Benchmark for Deep Learning on Relational Databases. arXiv:2407.20060.
- Weytjens, H.; and Weerdt, J. D. 2021. Creating Unbiased Public Benchmark Datasets with Data Leakage Prevention for Predictive Process Monitoring. *CoRR*, abs/2107.01905.

Yang, C.; Brower-Sinning, R. A.; Lewis, G. A.; and Kästner, C. 2022. Data Leakage in Notebooks: Static Detection and Better Processes. [arXiv:2209.03345](https://arxiv.org/abs/2209.03345).